# EXAMSHIELD: OCR-BASED IDENTITY VERIFICATION AND EXAM AUTHENTICATION SYSTEM

## SCRUM BOOK: SUBMISSION - 2 (CIE 2)

### STUDENT PROFILE

- **Name of Student:** ATHUL KRISHNA R
- **Register Number:** MEA24MCA-XXXX
- **Course & Batch:** Master of Computer Applications (MCA)
- **Semester:** S3 (Third Semester)
- **Department:** Department of Computer Applications
- **College:** MEA Engineering College, Perinthalmanna

### EVALUATION PROFILE

- **Submission Date:** 18/08/2026
- **Evaluation Event:** Continuous Internal Evaluation 2 (CIE 2)
- **Project Guide:** Ms. Prajina K (Assistant Professor)
- **Project Co-ordinator:** Mrs. Sruti Sudevan (HOD & Assistant Professor)

### EVALUATION & SIGNATURE SHEET

| Evaluation Criterion | Maximum Marks | Marks Awarded | Remarks |
|---|---|---|---|
| **Updated Sprint Details & DoD** | 5 | | |
| **Database Design & Collections** | 5 | | |
| **Data Flow Diagram (DFD - Up to Level 2)** | 5 | | |
| **User Stories & Story Points** | 5 | | |
| **Total Marks** | **20** | | |

**Signature of Project Guide:** _____

**Date of Evaluation:** _____

# 1. UPDATED SPRINT DETAILS

The **ExamShield** platform was developed following the **Agile Scrum Framework** across four sprints. Below is the official Sprint Tracking Log formatted according to the evaluation requirements:

### 🗓️ Sprint - 1

| Module | Task | Pending task if any | Hours of Completion | Expected date of Completion | Actual date of Completion | Reason for delay |
|---|---|---|---|---|---|---|
| **Security Staff** | Email-OTP Registration | - | 2 hr | 14/07/2026 | 14/07/2026 | - |
| **Security Staff** | JWT - Login & Role Auth | - | 2 hr | 16/07/2026 | 16/07/2026 | - |
| **Admin** | Master Admin Seeding Script | - | 1 hr | 17/07/2026 | 17/07/2026 | - |
| **Core** | Express & Mongoose Server Setup | - | 3 hr | 18/07/2026 | 18/07/2026 | - |
| **Core** | Socket.io Real-Time Event Server | - | 3 hr | 21/07/2026 | 21/07/2026 | - |
| **Core** | Protected Route Middleware (protect, authorize) | - | 2 hr | 24/07/2026 | 24/07/2026 | - |

### 🗓️ Sprint - 2

| Module | Task | Pending task if any | Hours of Completion | Expected date of Completion | Actual date of Completion | Reason for delay |
|---|---|---|---|---|---|---|
| **Student** | Multer Drag-Drop ID Photo Upload | - | 3 hr | 26/07/2026 | 26/07/2026 | - |
| **Student** | Student Registration & Profile API | - | 3 hr | 28/07/2026 | 28/07/2026 | - |
| **Student** | Bulk Student CSV Import & Parser | - | 4 hr | 30/07/2026 | 30/07/2026 | - |
| **Admin** | Student Master CRUD & Filter UI | - | 3 hr | 01/08/2026 | 01/08/2026 | - |
| **Student** | Digital Hall Ticket & Exam Card View | - | 2 hr | 03/08/2026 | 03/08/2026 | - |

## 🗓️ Sprint - 3

| Module | Task | Pending task if any | Hours of Completion | Expected date of Completion | Actual date of Completion | Reason for delay |
|---|---|---|---|---|---|---|
| **Verifier** | Tesseract.js OCR Engine Integration | - | 5 hr | 05/08/2026 | 05/08/2026 | - |
| **Verifier** | Live Gate Scanner & Camera Feed UI | - | 4 hr | 07/08/2026 | 07/08/2026 | - |
| **Verifier** | Text Extraction & Regex Normalization | - | 3 hr | 09/08/2026 | 09/08/2026 | - |
| **Verifier** | Multi-Field Match Scoring Algorithm | - | 3 hr | 11/08/2026 | 11/08/2026 | - |
| **Verifier** | Verification Decision & Rejection Flow | - | 4 hr | 13/08/2026 | 13/08/2026 | - |
| **Audit Log** | Verification Log Entry Persistence | - | 2 hr | 14/08/2026 | 14/08/2026 | - |

## 🗓️ Sprint - 4

| Module | Task | Pending task if any | Hours of Completion | Expected date of Completion | Actual date of Completion | Reason for delay |
|---|---|---|---|---|---|---|
| **Admin** | Real-Time Incident Feed (Socket.io) | - | 3 hr | 15/08/2026 | 15/08/2026 | - |
| **Admin** | Verification Stats & Distribution Charts | - | 2 hr | 16/08/2026 | 16/08/2026 | - |
| **Admin** | Comprehensive Verification History Table | - | 4 hr | 17/08/2026 | 17/08/2026 | - |
| **Admin** | CSV Audit Trail Report Export | - | 3 hr | 18/08/2026 | 18/08/2026 | - |

## 📊 Sprint Velocity & Burn-down Summary

| Sprint | Module Focus | Planned SP | Completed SP | Velocity Status |
|---|---|---|---|---|
| **Sprint 1** | Security Staff Auth & System Setup | 19 | 19 | On Target |
| **Sprint 2** | Student Master Data & ID Photo Uploads | 20 | 20 | On Target |
| **Sprint 3** | Tesseract OCR Scan Engine & Gate Verification | 21 | 21 | On Target |
| **Sprint 4** | Real-Time Security Dashboard & Audit Logs | 18 | 18 | On Target |

**Definition of Done (DoD) Checklist:**

☑ Code adheres to clean architecture with standard Express routes and React hooks.

☑ Password fields are hashed with Bcrypt (cost factor 12) prior to persistence.

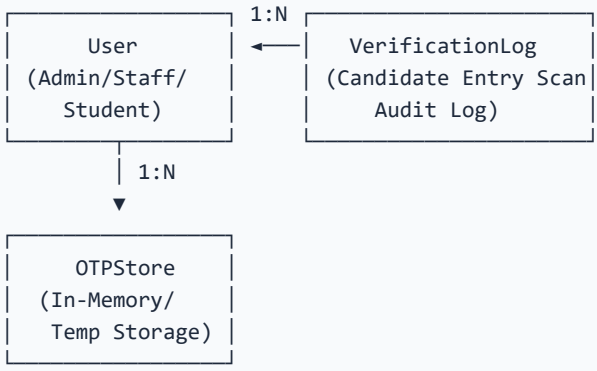
☑ API routes enforce JWT authorization middleware ( `protect` , `authorize` ).

☑ Tesseract OCR engine accurately extracts text and validates candidate roll numbers.

☑ All manual UAT workflows (Student, Admin, Security Staff) pass verification tests without runtime exceptions.

## 2. DATABASE DESIGN / COLLECTIONS

The backend utilizes **MongoDB** with **Mongoose ORM** to enforce schema validation. The primary collections and their attributes are detailed below:




### 2.1 Collection: users ( `User.js` )

Stores user profiles for Students, Security Staff (Verifiers), and Administrators.

| Field | Type | Constraints | Description |
|---|---|---|---|
| `_id` | ObjectId | Primary Key, Auto | Unique user identifier |
| `name` | String | Required | Full name of the user |
| `email` | String | Required, Unique, Lowercase | Email address for login |
| `password` | String | Required, Min length 6 | Hashed password (Bcrypt) |
| `role` | String | Enum: `['admin', 'security_staff', 'student']` | User access role |
| `phone` | String | Match: 10 digits, Default: null | Contact phone number |
| `studentProfile` | Object | Populated if `role === 'student'` | Student exam details |
| `isActive` | Boolean | Default: `true` | Account active status |
| `createdBy` | ObjectId | Ref: `'User'`, Default: null | Admin who created account |

**Sample Document ( users ):**

```
{
  _id: ObjectId('6a6959be4b8992b198eddbc0'),
  name: 'Athul Krishna R',
  email: 'athul98517772@gmail.com',
  password: '$2a$12$w8.9Xj1s4rZ6Qk9Y... (Bcrypt Hashed)',
  phone: '9876543210',
  role: 'student',
  studentProfile: {
    rollNumber: 'MEA24MCA001',
    course: 'Master of Computer Applications (MCA)',
    semester: 3,
    examCenter: 'Main Block - Room 102',
    department: 'Computer Applications',
    admissionYear: 2024,
    dob: '2002-05-14',
    idNumber: 'AADHAAR-8921-4567',
    examSubject: 'Advanced Web Technologies',
    examDate: '2026-08-18',
    examTime: '10:00 AM - 01:00 PM',
    idCardImageUrl: '/uploads/student_cards/card_mea24mca001.png'
  },
  isActive: true,
  createdBy: ObjectId('6a6959be4b8992b198eddba1'),
  createdAt: ISODate('2026-07-28T10:15:30.000Z'),
  updatedAt: ISODate('2026-07-29T01:39:10.000Z'),
  __v: NumberInt('0')
}
```

### 2.2 Collection: verificationlogs ( `VerificationLog.js` )

Stores audit logs for candidate scan attempts, extracted OCR text, match scores, and entry gate decisions.

| Field | Type | Constraints | Description |
|---|---|---|---|
| `_id` | ObjectId | Primary Key, Auto | Unique log record ID |
| `student` | ObjectId | Ref: `'User'`, Default: null | Matched candidate reference |
| `scannedRollNumber` | String | Trim, Default: `""` | OCR extracted roll number |
| `scannedName` | String | Trim, Default: `""` | OCR extracted candidate name |
| `rawOcrText` | String | Default: `""` | Full raw OCR text output |
| `matchedDetails` | Object | Sub-fields: name, rollNumber, course | Snapshot of DB match info |
| `matchPercentage` | Number | Required, Range: 0 to 100 | Fuzzy match accuracy score |
| `status` | String | Enum: `['APPROVED', 'REJECTED']` | Entry gate decision status |
| `rejectionReason` | String | Trim, Default: `""` | Reason for entry denial |
| `idCardPhotoPath` | String | Required | Path to uploaded scan image |
| `verifiedBy` | ObjectId | Required, Ref: `'User'` | Security staff / Verifier ID |

📄 **Sample Document ( verificationlogs ):**

```
{
  _id: ObjectId('6a6959be4b8992b198eddbd2'),
  student: ObjectId('6a6959be4b8992b198eddbc0'),
  scannedRollNumber: 'MEA24MCA001',
  scannedName: 'ATHUL KRISHNA R',
  rawOcrText: 'MEA ENGINEERING COLLEGE PERINTHALMANNA \nNAME: ATHUL KRISHNA R \nROLL NO: MEA24MCA001
\nCOURSE: MCA',
  matchedDetails: {
    name: 'Athul Krishna R',
    rollNumber: 'MEA24MCA001',
    course: 'Master of Computer Applications (MCA)',
    examCenter: 'Main Block - Room 102'
  },
  matchPercentage: 95,
  status: 'APPROVED',
  rejectionReason: '',
  idCardPhotoPath: '/uploads/scan_proofs/scan_20260818_100122.png',
  verifiedBy: ObjectId('6a6959be4b8992b198eddbbc'),
  createdAt: ISODate('2026-08-18T04:31:22.000Z'),
  updatedAt: ISODate('2026-08-18T04:31:22.000Z'),
  __v: NumberInt('0')
}
```

### 2.3 Collection: otps ( `otpStore.js / Otp` )

Temporary storage for One-Time Passwords during user registration and password resets.

| Field | Type | Constraints | Description |
|---|---|---|---|
| `_id` | ObjectId | Primary Key, Auto | Unique OTP record ID |
| `email` | String | Required | Target user email |
| `otp` | String | Required | 6-digit verification code |
| `name` | String | Required | Temporary registration name |
| `password` | String | Required | Temporary password string |
| `createdAt` | Date | Default: Date.now , Expires: 300s (5m) | Auto-deleted TTL index |

📄 **Sample Document ( otps ):**

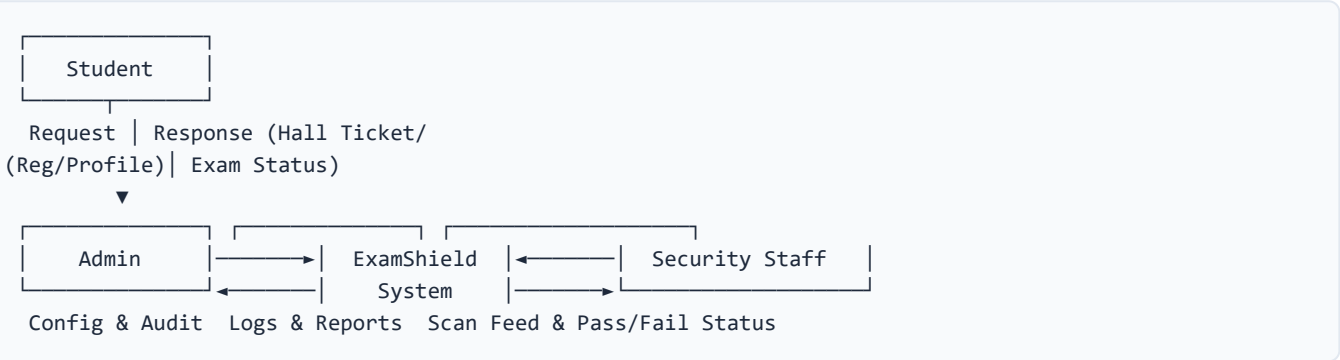
```
{
  _id: ObjectId('6a6959be4b8992b198eddbf0'),
  email: 'security.officer@examshield.org',
  otp: '482910',
  name: 'Gate Officer 1',
  password: '$2a$12$e7.2Wk3s...',
  createdAt: ISODate('2026-07-29T01:35:00.000Z')
}
```

# 3. DATA FLOW DIAGRAM (DFD)

The Data Flow Diagrams (DFD) represent the logical movement of data throughout the **ExamShield OCR-Based Identity Verification System** across Level 0, Level 1 (Role-specific), and Level 2 detailed process decompositions.

## 3.1 Level 0 - System Context Diagram

The Level 0 Context Diagram depicts the primary boundary of the ExamShield system and its data flow exchanges with External Entities (Student, Security Staff / Verifier, Municipal/Exam Admin).



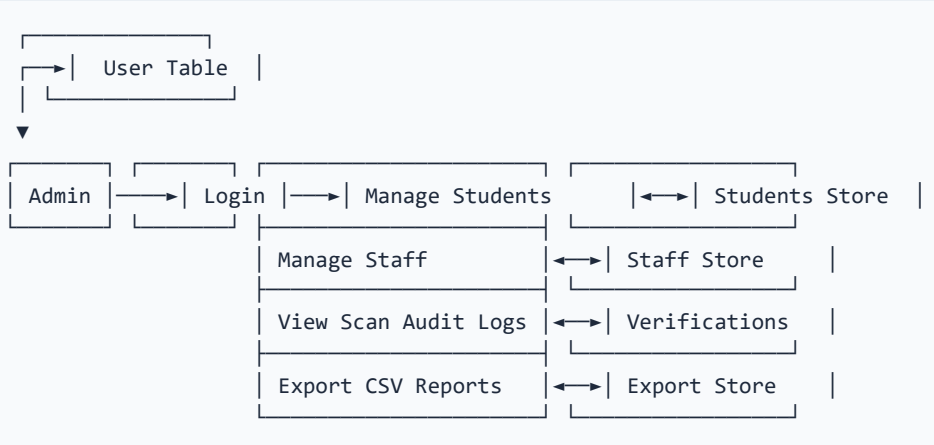

```
graph TD
    Student[👤 Student] -->|Register, Login & View Hall Ticket| System((🌐 ExamShield Platform))
    System -->|Exam Center, Roll Info & Status| Student

    Admin[🛡️ System Admin] -->|Student CSV Batch, Staff Accounts & Rules| System
    System -->|Audit Logs, Verification Statistics & Analytics| Admin

    Staff[👮 Security Staff] -->|ID Photo Scan Upload & Gate Request| System
    System -->|OCR Extract Match Result, Pass/Fail Decision| Staff
```

## 3.2 Level 1 - Admin

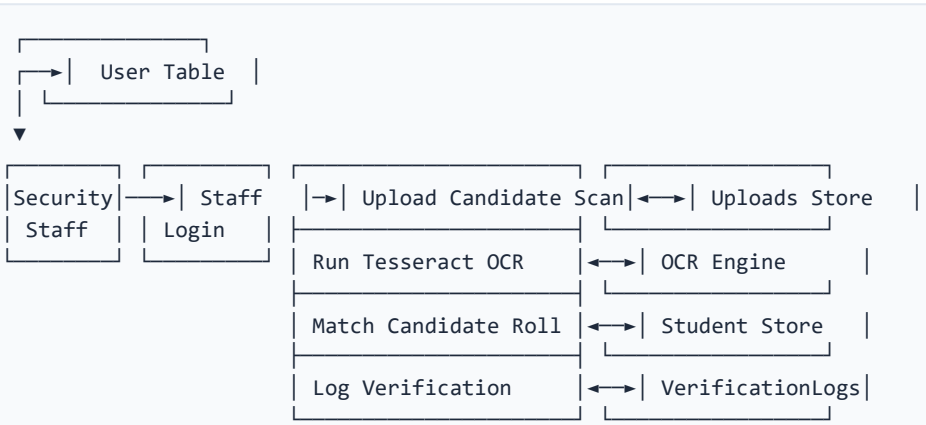
Decomposes administrative functionality. The Admin authenticates via the User Table and manages students, verifiers, scan logs, analytics, and CSV exports.



```
graph TD
    Admin[🛡️ Admin] --> Login((Login))
    Login <--> UserTable[(📁 User Table)]
    Login --> P1((Manage Students)) <--> D1[(📁 Student Profiles Store)]
    Login --> P2((Manage Security Staff)) <--> D2[(📁 Staff Accounts Store)]
    Login --> P3((Audit Scan History)) <--> D3[(📁 Verification Logs Store)]
    Login --> P4((Export CSV Reports)) <--> D3
```

### 3.3 Level 1 - Security Staff (Verifier)

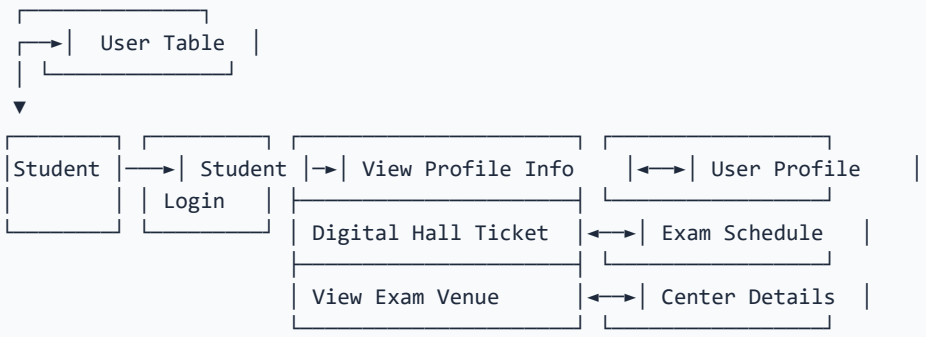
Decomposes gate verifier functionality. Security officers log in, capture/upload candidate ID cards, execute Tesseract OCR scanning, match extracted details against database records, and issue pass/fail decisions.




```
graph TD
    Staff[👮 Security Staff] --> SLogin((Staff Login))
    SLogin <--> UserTable[(📁 User Table)]
    SLogin --> SP1((Upload ID Photo Scan)) <--> SD1[(📁 Uploads Store)]
    SLogin --> SP2((Execute OCR Parsing)) <--> SD2[(📁 Tesseract Engine)]
    SLogin --> SP3((Query Roll Number Match)) <--> SD3[(📁 Student Store)]
    SLogin --> SP4((Save Gate Decision & Emit Alert)) <--> SD4[(📁 Verification Logs)]
```

### 3.4 Level 1 - Student

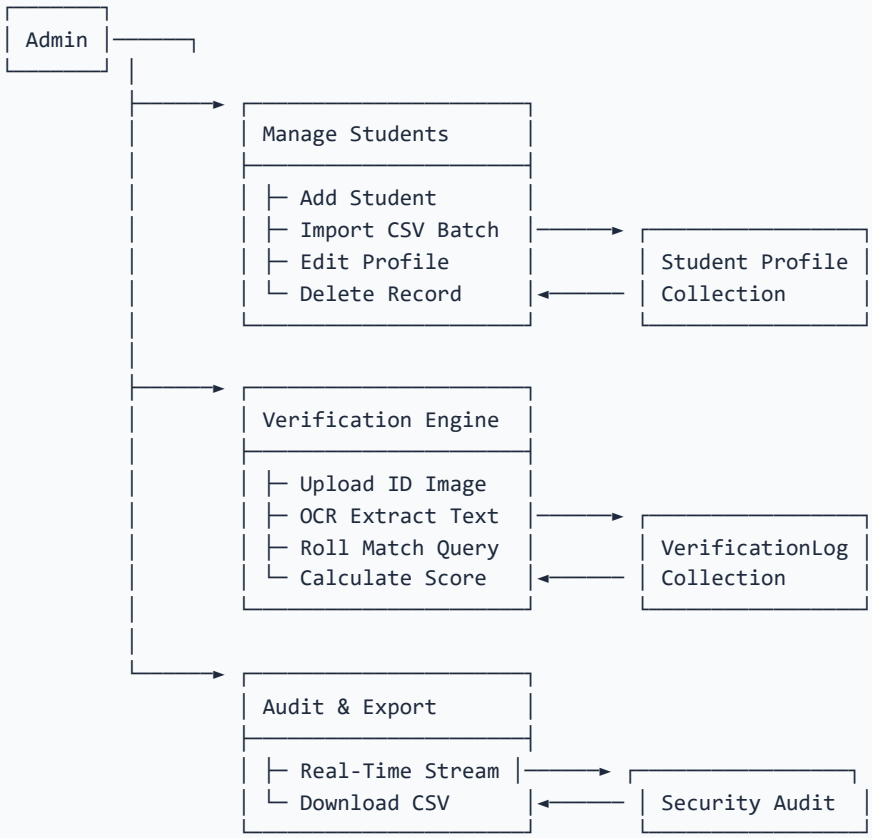
Decomposes student functionality. Registered candidates authenticate to access digital hall tickets, view assigned exam centers and seating schedules, and verify registration status.



```
graph TD
    Student[👤 Student] --> StLogin((Student Login))
    StLogin <--> UserTable[(📁 User Table)]
    StLogin --> StP1((View Candidate Profile)) <--> StD1[(📁 Users Store)]
    StLogin --> StP2((Generate Digital Hall Ticket)) <--> StD2[(📁 Exam Schedule Store)]
    StLogin --> StP3((View Gate Scan History)) <--> StD3[(📁 Verification Logs)]
```

### 3.5 Level 2 - Verification Engine & Admin (Detailed Decomposition)

Detailed Level 2 breakdown showing exact sub-processes and datastores managed by the Verification Engine and Administrator.




```
graph TD
    Admin[🛡️ Admin] --> Students[Manage Students]
    Students --> W1((Add Student)) --> D1[(📁 User Store)]
    Students --> W2((Import CSV Batch)) <--> D1
    Students --> W3((Edit Profile)) --> D1
    Students --> W4((Delete Student)) --> D1

    Staff[👮 Verifier] --> Engine[Verification Engine]
    Engine --> E1((Upload Scan Photo)) --> D2[(📁 Uploads)]
    Engine --> E2((Run Tesseract OCR)) --> E3((Extract Roll Number))
    E3 --> E4((Match Against DB)) <--> D1
    E4 --> E5((Compute Match % & Status)) --> D3[(📁 Verification Logs)]
    E5 --> E6((Emit Socket.io Event))
```

## 4. USER STORIES

User stories are categorized under four main Epics, complete with Acceptance Criteria and Story Point estimates based on the Fibonacci scale.

### Epic 1: Identity & Access Management (IAM)

#### 📖 User Story 1.1: Security Staff & Admin Registration with Email OTP

**As a** Security Officer / Administrator,
**I want to** register an account using my email and verify via a 6-digit OTP,
**So that** only authorized gate personnel can log into the verification portal.
**Acceptance Criteria:**

- Nodemailer dispatches a 6-digit OTP code with 5-minute TTL expiration.
- Passwords are pre-hashed using Bcrypt (12 salt rounds) prior to storage.
- Returns HTTP `201 Created` upon successful OTP validation.

**Story Estimate:** 3 SP

#### 📖 User Story 1.2: Unified JWT Authentication & Role Control

**As a** User (Student / Admin / Security Staff),
**I want to** authenticate using my registered credentials,
**So that** I can receive a secure JSON Web Token and access role-restricted pages.
**Acceptance Criteria:**

- Returns JWT token containing user ID and role payload on valid match.
- Invalid credentials return HTTP `401 Unauthorized` response.
- Frontend stores JWT in `localStorage` and auto-attaches via Axios headers.

**Story Estimate:** 3 SP

### Epic 2: Student Master Data & ID Management (SMD)

#### 📖 User Story 2.1: Student Profile Creation & ID Card Photo Upload

**As an** Administrator,
**I want to** add candidate details and upload their reference ID card photo,
**So that** candidate records exist for gate matching during examinations.
**Acceptance Criteria:**

- Multer middleware handles image upload and saves path to student profile.
- Enforces unique roll number constraint in MongoDB database.

**Story Estimate:** 5 SP

#### 📖 User Story 2.2: Bulk Student Registration via CSV Import

**As an** Administrator,
**I want to** upload a CSV file containing candidate records,
**So that** I can onboard thousands of exam candidates simultaneously.
**Acceptance Criteria:**

- Parses CSV rows into structured JSON objects and batch-inserts to MongoDB.
- Returns clear breakdown of successfully imported versus duplicate records.

**Story Estimate:** 4 SP

## Epic 3: OCR Gate Verification & Identity Matching (OIV)

### 📖 User Story 3.1: Live Camera Scan & Image Capture UI

**As a** Security Staff Verifier,
**I want to** capture or upload a candidate's ID card photo at the entrance gate,
**So that** the system can process the image for text recognition.
**Acceptance Criteria:**

- Webcam integration allows instantaneous frame capture or drag-drop upload.
- Visual loading overlay displays scan progress to the officer.

**Story Estimate:** 5 SP

### 📖 User Story 3.2: Tesseract OCR Engine Text Extraction & Matching

**As a** Security Staff Verifier,
**I want** Tesseract OCR to extract roll numbers and candidate names automatically,
**So that** manual keying errors are eliminated during high-volume entry hours.
**Acceptance Criteria:**

- Regex normalizes extracted text to isolate candidate roll numbers.
- Queries MongoDB for matching student profile and computes match score.

**Story Estimate:** 5 SP

## Epic 4: Security Dashboard & Real-Time Analytics (SDA)

### 📖 User Story 4.1: Real-Time Socket.io Security Scan Feed

**As an** Administrator,
**I want to** view live gate scan results as they occur across all entry gates,
**So that** I can immediately spot fraudulent attempts or impersonation alerts.
**Acceptance Criteria:**

- Socket.io pushes scan events instantly to the central admin dashboard.
- Displays green APPROVED and red REJECTED badges in real time.

**Story Estimate:** 5 SP

## 5. GIT COMMIT HISTORY SUMMARY (REVIEW CYCLE)

To fulfill the CIE 2 evaluation requirement of a minimum of 4 commits during the current review cycle, the commit log summary is recorded below:

```
commit 618f7f58e61ac703860eaf94d8420c1842996689
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Wed Jul 29 22:01:16 2026 +0530

    fixed

commit 588bab2a73f817ce41e6bbdcdc7126d9fe1f64ad
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Wed Jul 29 19:14:44 2026 +0530

    nodemailer otp,reg setup

commit c1b758c401cfcb53a672a0742c8be3b2292b1b12
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Mon Jul 27 22:21:40 2026 +0530

    add

commit f377175e742e6d5651c7e22af9185e7d322e6849
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Mon Jul 27 21:29:47 2026 +0530

    fixed

commit d66287037bcf86a7500e60e4e6a8739244b5f08b
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Wed Jul 15 16:10:43 2026 +0530

    color changed

commit 71359b1c3d7c8a856a6e48e4f2fffe6d85ca7440
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Tue Jul 14 16:37:55 2026 +0530

    verification log

commit ffa0c6ad3a003357d939dacb3002c2d93a2723ef
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Tue Jul 14 14:47:11 2026 +0530

    multer config

commit cf18405619fb3aafb4abdd49350e5f92ce81045b
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Tue Jul 14 00:04:28 2026 +0530

    auth

commit 606690fe3ee3021baecc81d215cc43448e25d9b1
Author: Athul3588krishna <athul98517772@gmail.com>
Date:   Mon Jul 13 23:53:03 2026 +0530

    initialised
```